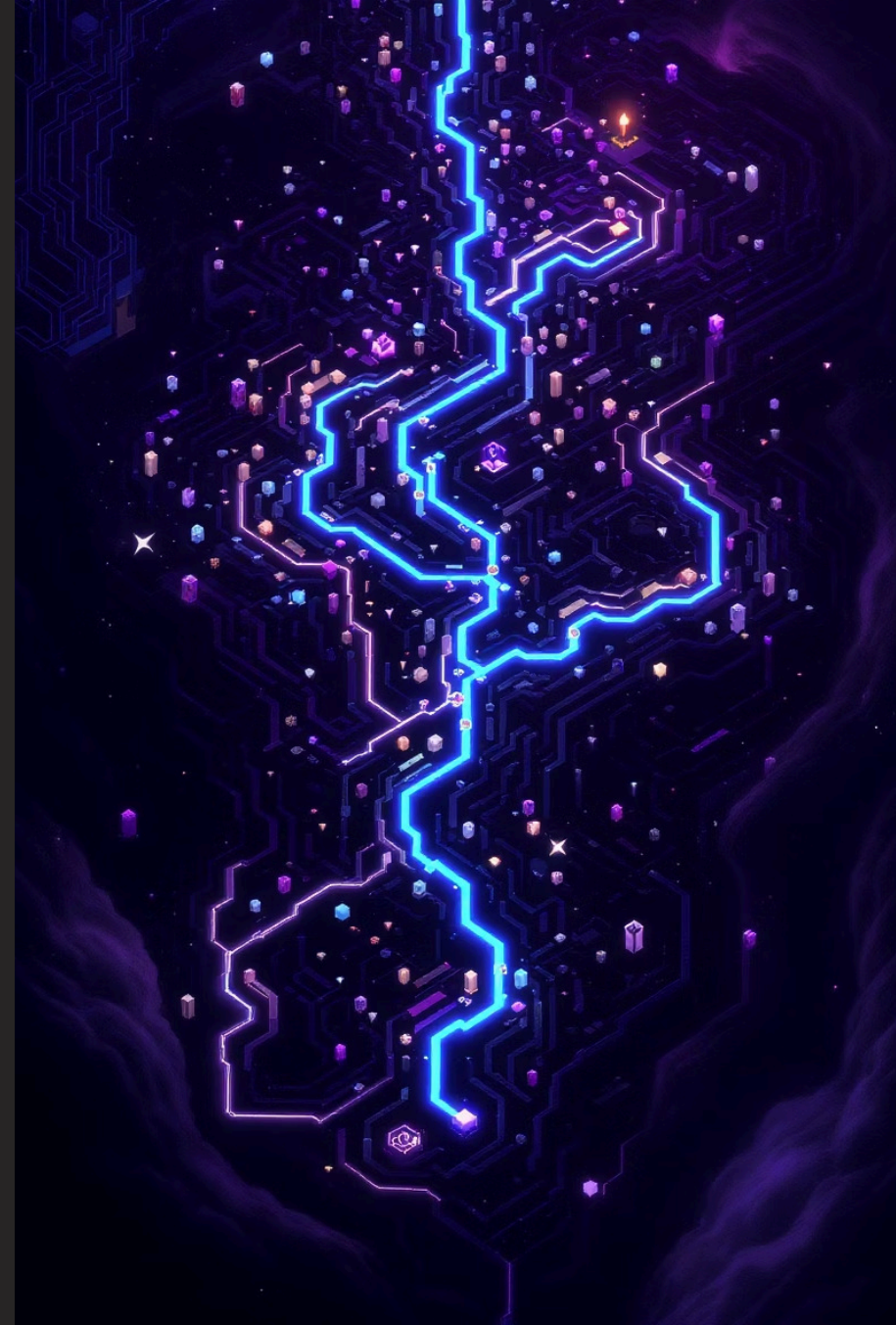


Large-Scale Path Planning in Maps: Research, Methods, and Applications

A comprehensive overview of current research, methodologies, and applications in large-scale pathfinding algorithms, with insights from leading researchers and practical implementations.



Research Landscape: Key Players and Milestones

1

Computer Science/Gaming Pioneers

Research began approximately 15 years ago, largely driven by gaming industry requirements.

- Adi Botea: HPA* (Hierarchical Pathfinding A*), CPD
- Nathan R. Sturtevant: PRA* family algorithms
- Daniel Harabor: Polyanya, JPS (Jump Point Search)
- Sven Koenig: Various contributions

2

Robotics Specialists

Adapting pathfinding algorithms for real-world robotic applications with unique constraints.

- Maxim Likhachev: MRA (Multi-Resolution A*), AMRA
- Specialized implementations: CFA* (Cooperative FA*)
- WaveStar: Wave propagation techniques

3

Distributed Computing Researchers

Focusing on parallelization and distribution of pathfinding workloads.

- Map-Reduce A* implementations
- Parallel processing techniques
- Cloud-based pathfinding solutions

Research identified through Google Scholar citation tracking, refined with "large map" queries, resulting in over 200 relevant publications.

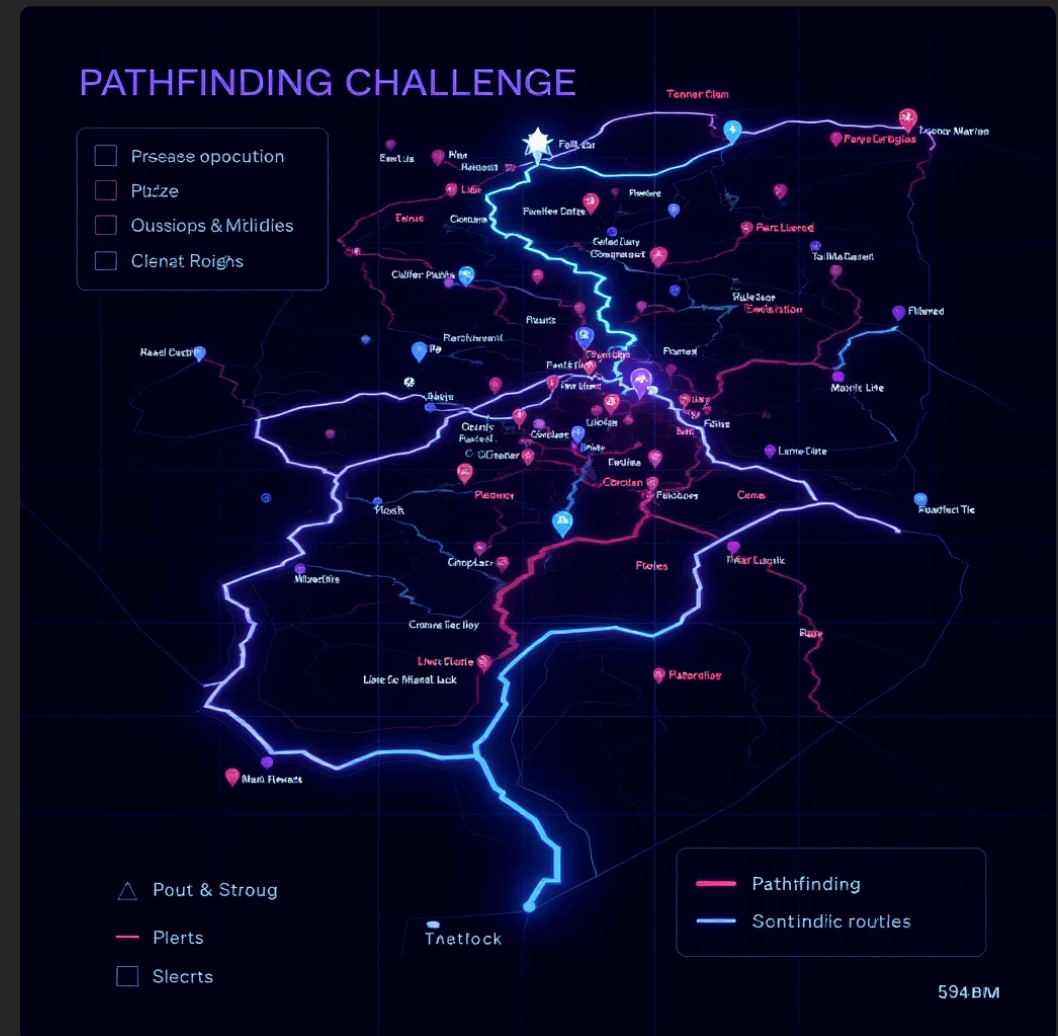
Current Benchmarks and Map Scales

MovingAI Dataset Dominance

Most current research relies on the MovingAI benchmark maps, which include:

- Maps from StarCraft
- Dragon Age: Origins environments
- Various grid-based game worlds

These maps have become the de facto standard for algorithm comparison and evaluation.



GPPC (Grid-Based Path Planning Competition) defines "big maps" as 1024×1024 grids, which represents the current upper bound for most published research.

Methodological Approaches



Heuristic Improvements

Enhancing distance estimation functions

- D* and variants
- Any-angle pathfinding
- Euclidean and Manhattan hybrid approaches



Hierarchical Abstraction

Reducing search space complexity

- HPA* (Hierarchical Pathfinding A*)
- JPS (Jump Point Search)
- Multi-level abstraction techniques



Engineering Optimizations

Implementation-level enhancements

- Memory management techniques
- Cache optimization
- Algorithmic refinements



Experimental Evaluations

Initial comparative testing was conducted using the python-motion-planning repository as a unified framework, evaluating seven distinct pathfinding methods:

- A* (baseline algorithm)
- D* (Dynamic A*)
- JPS (Jump Point Search)
- THETA* (Any-angle pathfinding)
- Bidirectional search variants
- Weighted A* implementations
- Hybrid approaches

Monte Carlo experiments revealed no statistically significant performance differences between these methods when implemented in Python, suggesting that algorithmic changes alone are insufficient for large-scale improvements.

Implementation Language Impact

10-50x

Speed Improvement

Switching from Python to C++/C# implementations yielded dramatic performance gains across all algorithms.

PyBind11

Integration Method

Python bindings allowed efficient utilization of native code performance while maintaining Python's flexibility for experimentation.

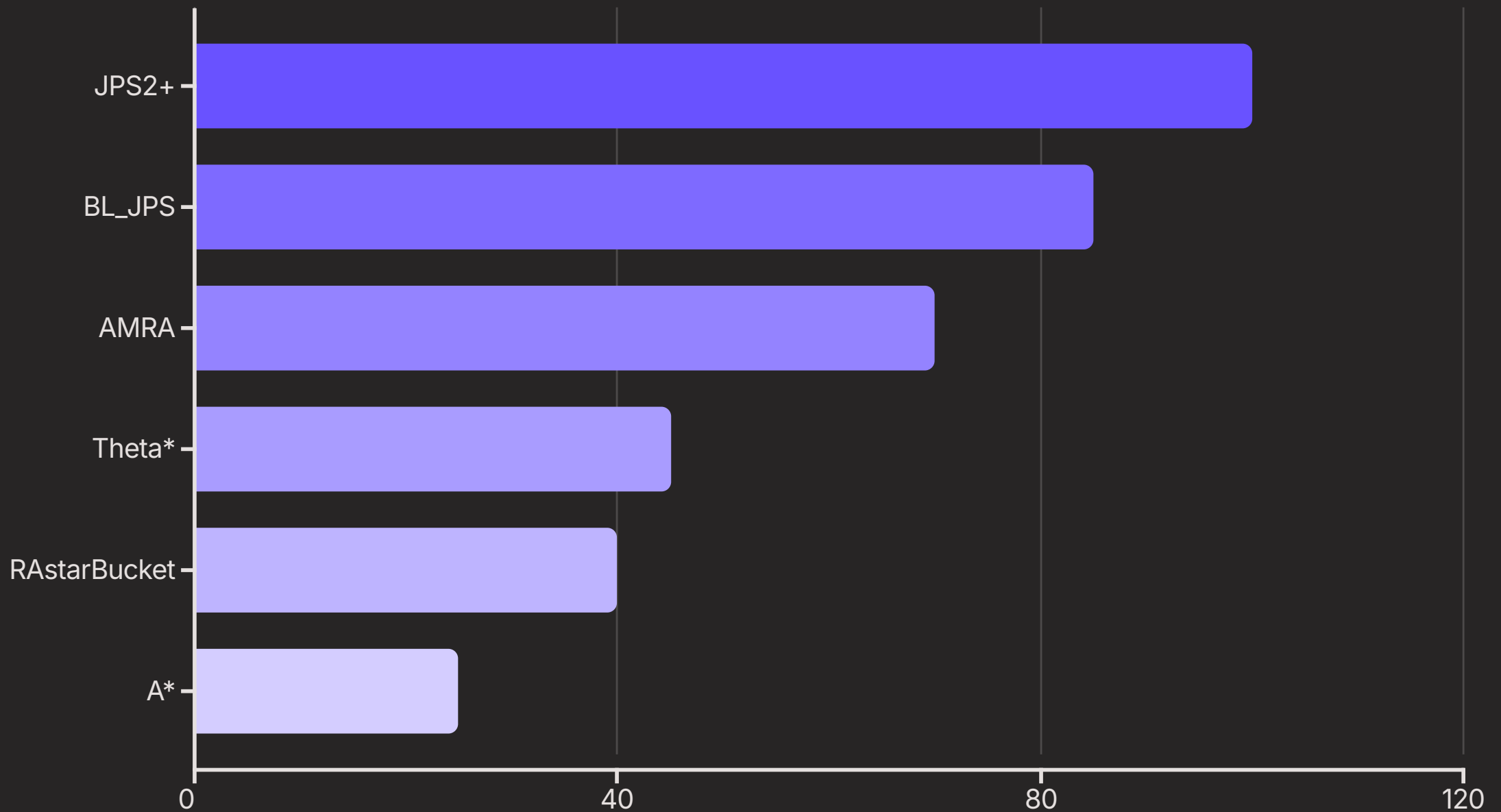
BL_JPS

Best Performer

Without precomputation, BL_JPS (Boundary Labeling Jump Point Search) demonstrated superior performance across test scenarios.

For precomputation-enabled scenarios, Warthog's JPS2+ algorithm showed the best overall performance, consistent with GPPC 2014 competition results.

Comparative Algorithm Performance



Algorithm implementations evaluated include: JPS_planner_bindings, ThetaStarPlanner, BL_JPS, Warthog, RAstarBucket, and AMRA. Results broadly confirm the findings from the GPPC 2014 competition, with hierarchical and jump-point based methods consistently outperforming traditional approaches.

Future Research Directions

Navigation Mesh Approaches

Implementing and evaluating dynamic NavMesh construction and updates using Python's ctypes for C++ library integration. This approach shows promise for handling large, complex environments with fewer memory requirements than grid-based methods.

Visibility Graph (VG) Techniques

Exploring visibility graph construction algorithms that can efficiently represent navigable space in large environments. This representation allows for optimal paths while reducing the computational complexity compared to grid-based approaches.

Hybrid Approaches

Combining the strengths of multiple algorithms: hierarchical decomposition for global planning, local techniques for fine-tuning, and precomputation strategies for runtime efficiency in large-scale scenarios.

Next steps include benchmarking these approaches on significantly larger maps (beyond the current 1024×1024 standard) and evaluating their performance in dynamic environments with changing obstacles.